

Node allocator benchmark

Page allocators vs. the default pool, for `boost::root_ptr`, `std::unique_ptr` and `std::shared_ptr`

Measured 2026-09-27 on an Intel Core i7-4700HQ (4 cores / 8 threads), Linux 5.15, clang 23 -02, libstdc++ 12, glibc 2.35. Median of 5 runs; every (pointer, allocator, scenario) triple runs in a fresh process. Full tables and method: `ALLOCATOR_BENCHMARK.md`.

Findings

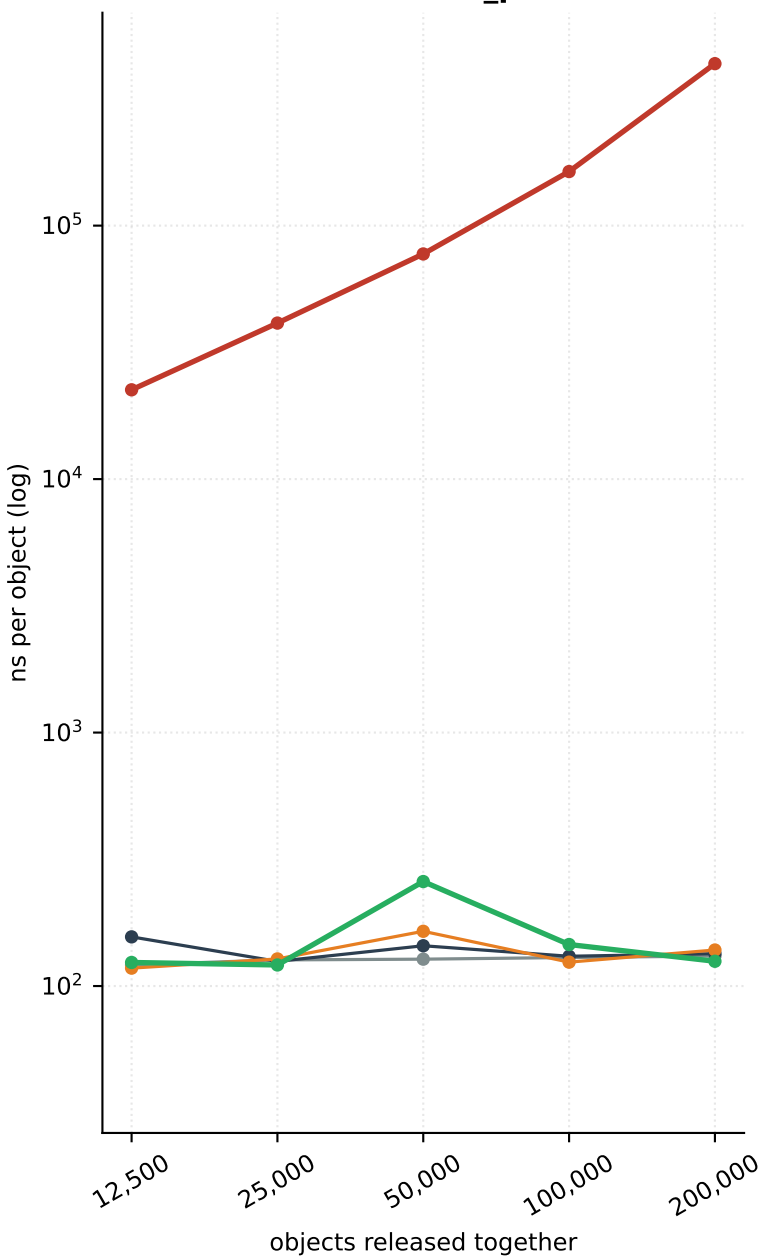
- The default `boost::pool_allocator` is quadratic on release: its `deallocate()` keeps the free list sorted (`ordered_free`), so releasing 200,000 objects costs 253-436 μ s per object. Every other allocator stays flat, at 40-160 ns per object.
- `page_allocator_by_size` is the fastest, or within 5% of it, in every single-threaded scenario, except churn with `unique_ptr` and `shared_ptr` (`std::allocator` 12-17% faster). It uses 9-18% less memory than the default pool.
- `page_allocator_by_type` costs 2.6-4x the time and 1.6-1.9 MB more memory than `by_size` with many sparse types: each type fills a 64 KiB page of its own.
- With four threads, `std::allocator` is 18-38x faster than any pool for `unique_ptr` and `shared_ptr`; for `root_ptr` the global mutex narrows the gap to 1.4x.
- `root_ptr` costs about 3x `unique_ptr` in time and 1.8x in memory; it alone reclaims cycles.

Pages

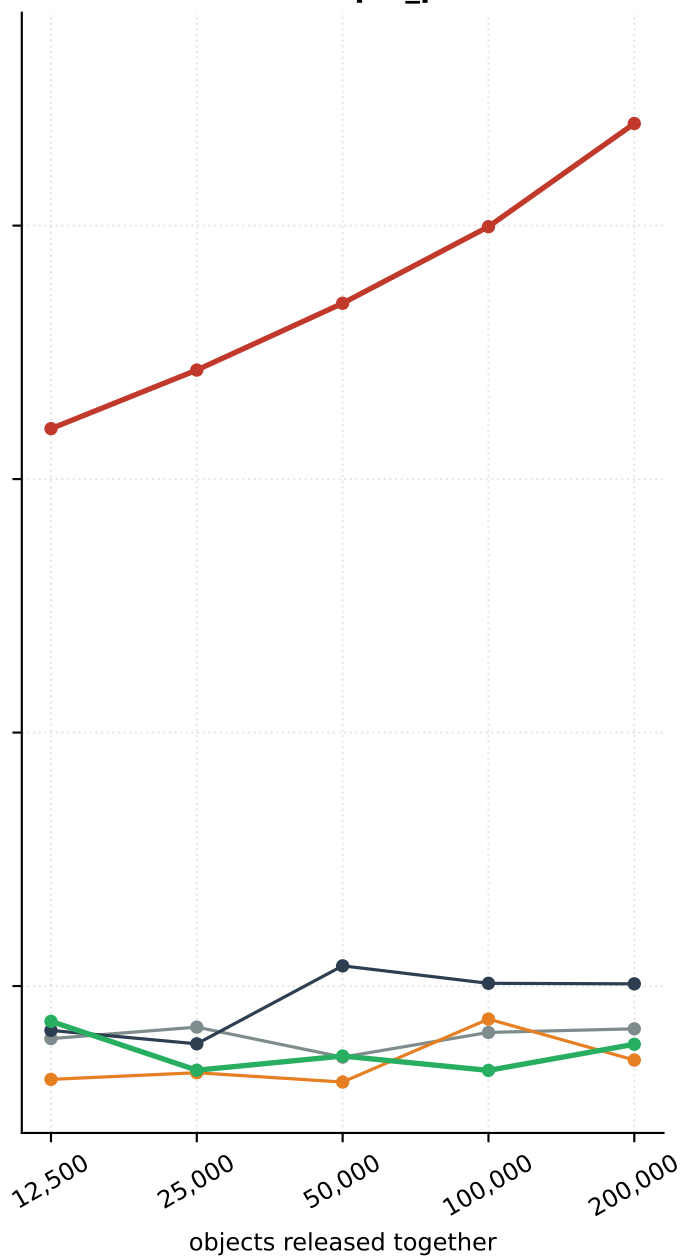
2	Bulk release scaling	3	Time per scenario	4	Resident memory
5	Pointer types compared	6	Threads		

Bulk release scaling: the default pool grows with the number of objects, the others stay flat

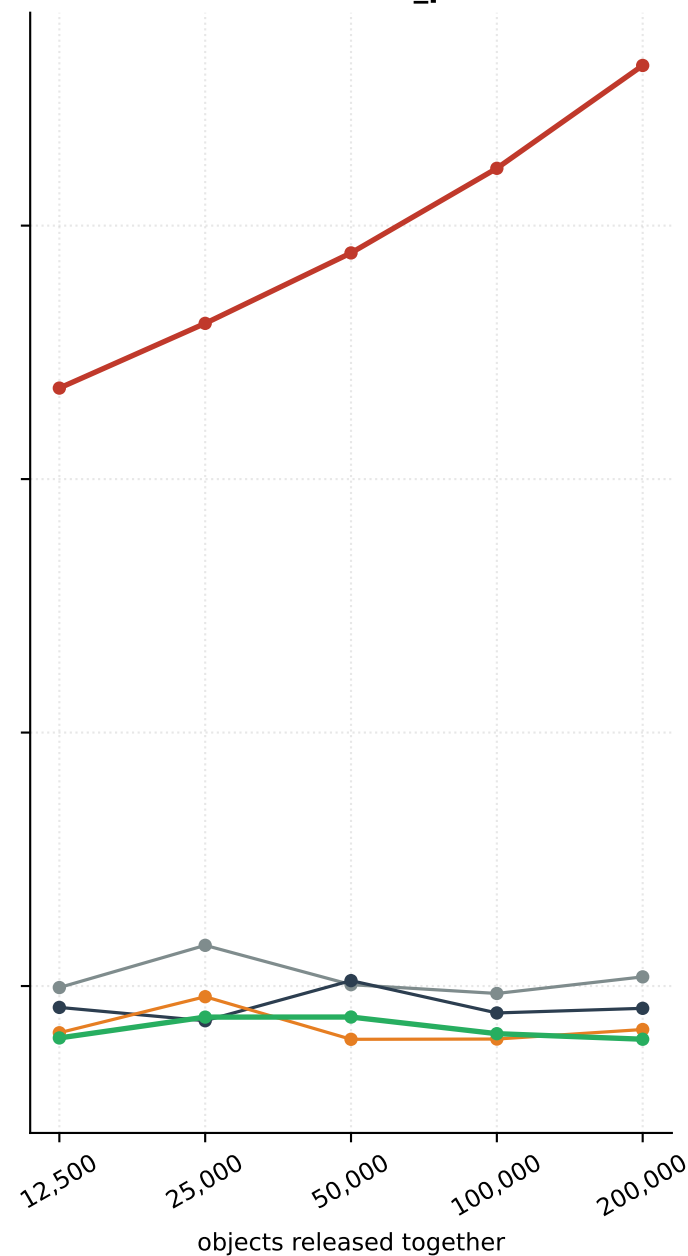
boost::root_ptr



std::unique_ptr



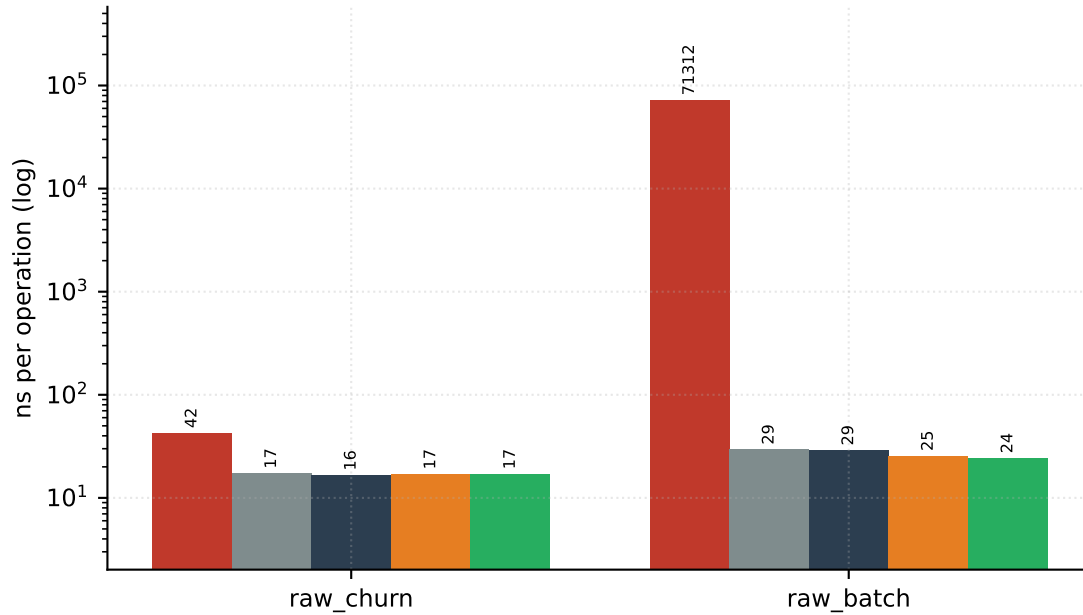
std::shared_ptr



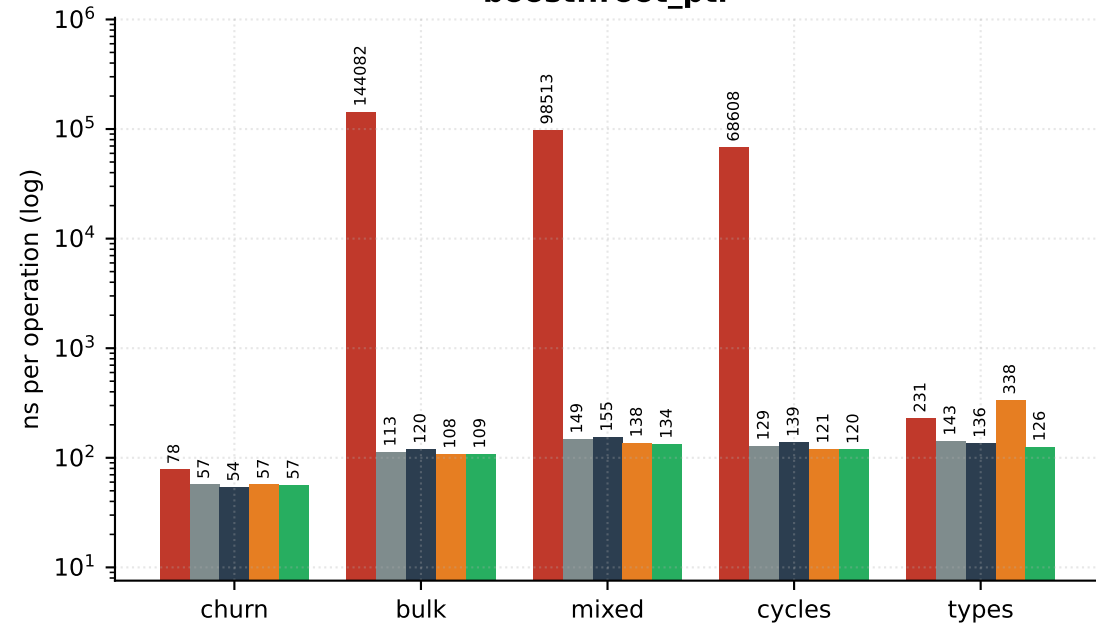
boost::pool_allocator (default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size

Time per operation, single-threaded (lower is better)

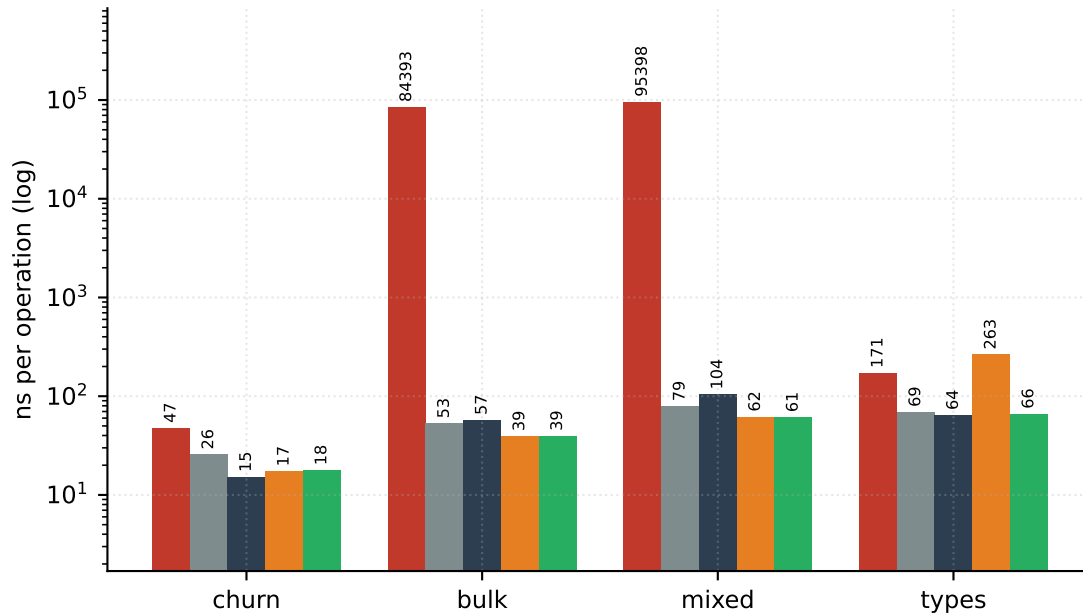
Allocator alone (no pointer)



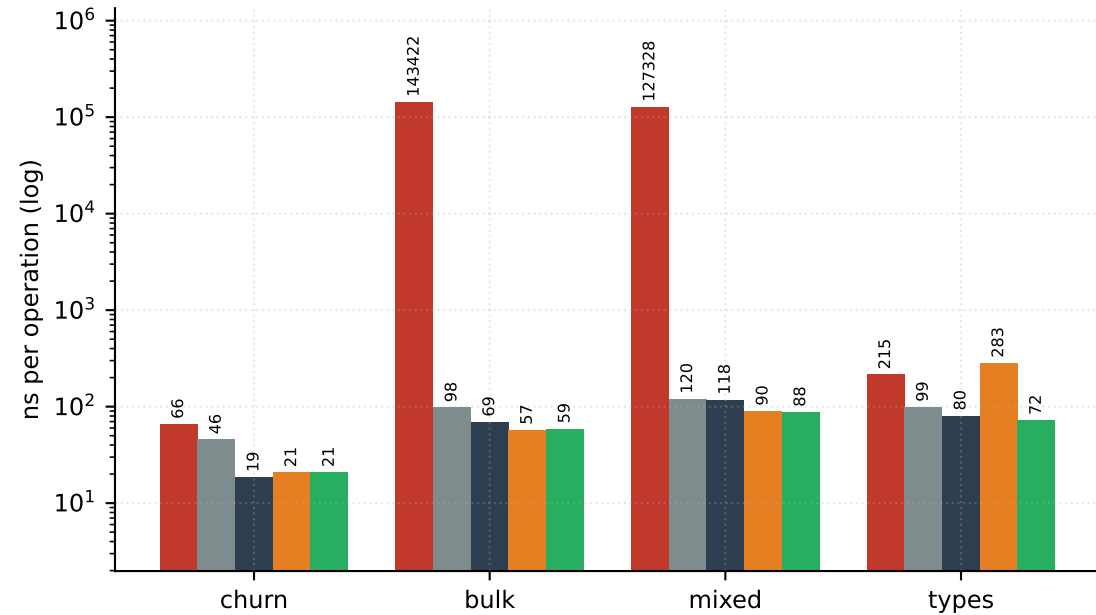
boost::root_ptr



std::unique_ptr



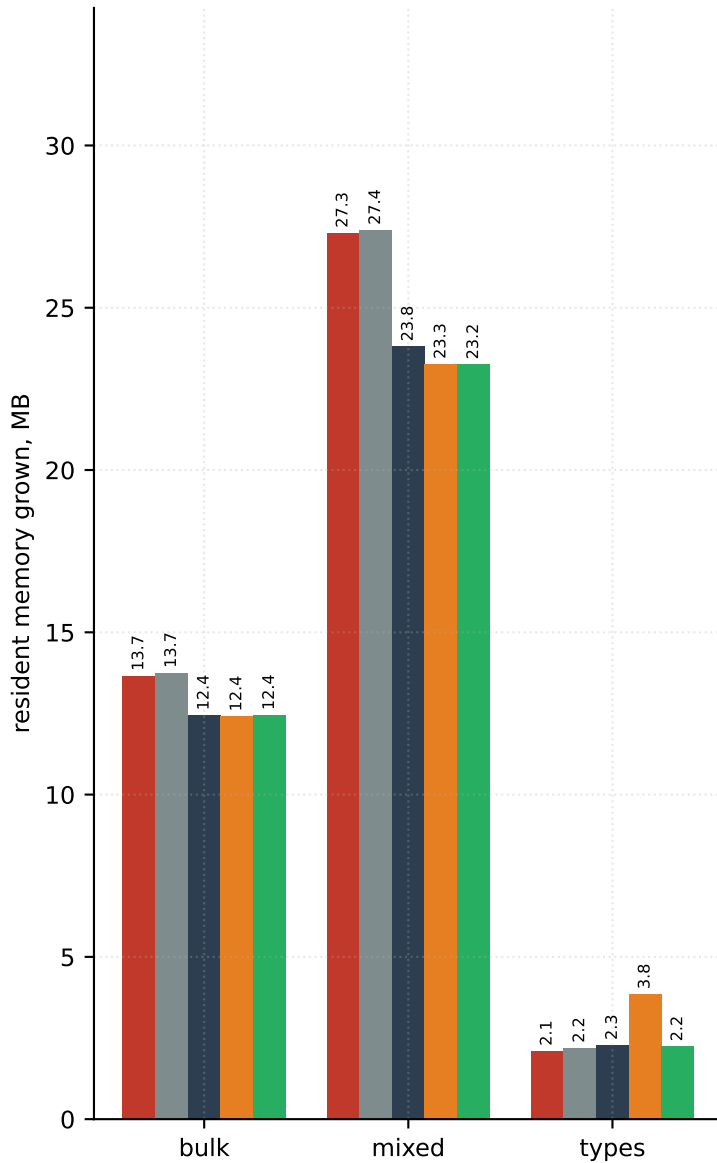
std::shared_ptr



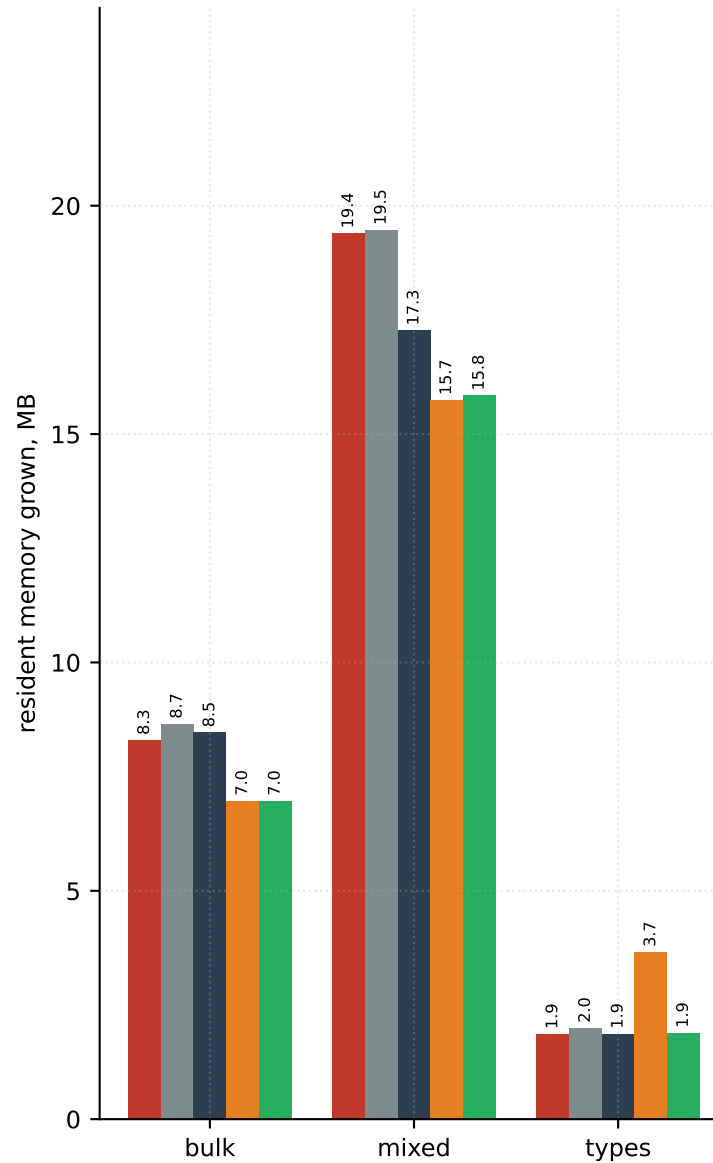
boost::pool_allocator (default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size

Resident memory while the objects are live (lower is better)
bulk: 100,000 x 48 B mixed: 50,000 each of 16, 48 and 200 B types: 32 types x 100 objects

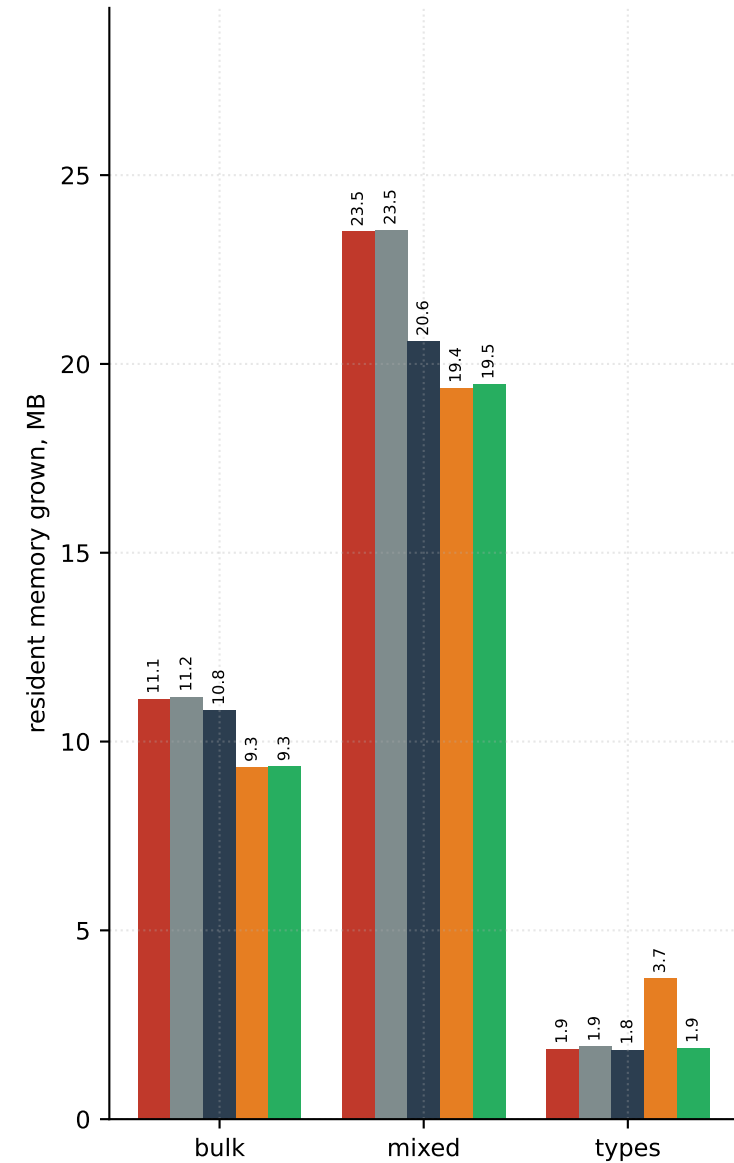
boost::root_ptr



std::unique_ptr



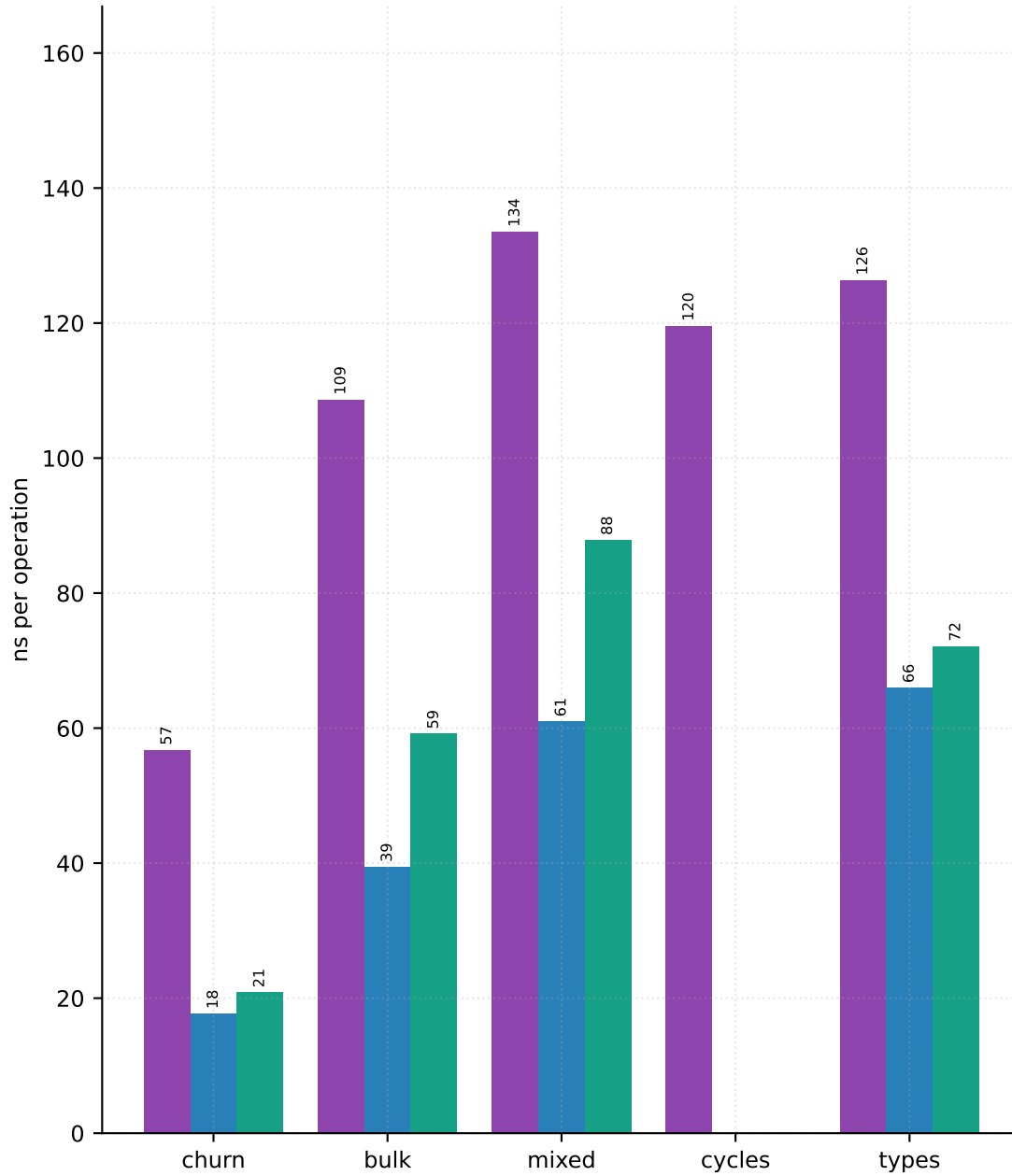
std::shared_ptr



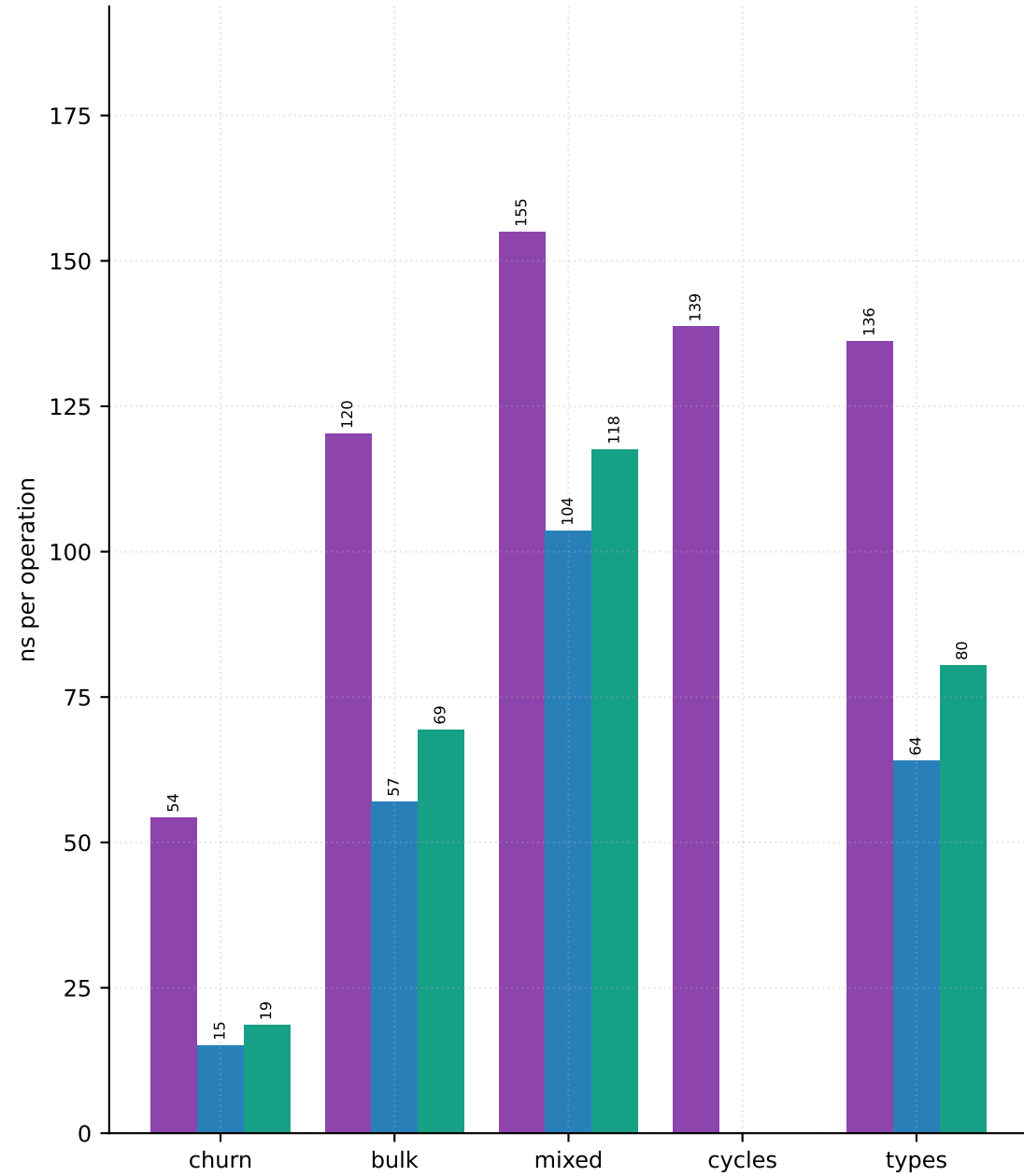
boost::pool_allocator (default) boost::fast_pool_allocator std::allocator page_allocator_by_type page_allocator_by_size

Pointer types compared (cycles: root_ptr only - a unique_ptr cannot form one, a shared_ptr cycle is never freed)

with page_allocator_by_size



with std::allocator



boost::root_ptr std::unique_ptr std::shared_ptr

4 threads, each creating and dropping 250,000 objects (lower is better)

glibc's per-thread caches take no lock; each page pool has one mutex; every root_ptr operation takes a global mutex

